

Python: Assignment

1) What are the types of Applications?

Common types of applications:

- Desktop Applications
- Web Applications
- Mobile Applications
- Enterprise Applications
- Embedded Applications

Cloud Applications

- AI/ML Applications
 - Gaming Applications
-

2) What is Programming?

Programming is the process of writing instructions (code) that a computer can understand and execute to perform specific tasks.

3) What is Python?

Python is a high-level, interpreted, object-oriented, and general-purpose programming language known for its simple syntax and readability.

4) Python program to check if a number is positive, negative, or zero

```
num = int(input("Enter a number: "))

if num > 0:
    print("Positive Number")
elif num < 0:
    print("Negative Number")
else:
    print("Zero")
```

5) Python program to find factorial of a number

```
num = int(input("Enter a number: "))
fact = 1

for i in range(1, num + 1):
    fact *= i

print("Factorial =", fact)
```

6) Python program to generate Fibonacci series:

```
n = int(input("Enter range: "))

a, b = 0, 1

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

7) How is memory managed in Python?

Python manages memory automatically using:

- Private Heap Space
- Memory Manager
- Garbage Collection
- Reference Counting

Unused objects are automatically removed from memory.

8) What is the purpose of continue statement in Python?

The continue statement skips the current iteration and moves to the next iteration of a loop.

Example:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output:

```
1
2
4
5
```

9) Swap two numbers with and without temporary variable

Using temporary variable

```
a = 10
b = 20

temp = a
a = b
b = temp

print(a, b)
```

Without temporary variable

```
a = 10
b = 20

a, b = b, a

print(a, b)
```

10) Python program to check whether a number is even or odd

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

11) Write a Python program to test whether a passed letter is a vowel or not.

```
letter = input("Enter a letter: ").lower()

if letter in "aeiou":
    print("Vowel")
else:
    print("Not a vowel")
```

12) Write a Python program to sum of three given integers. However, if two values are equal sum will be zero.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
```

```
c = int(input("Enter third number: "))

if a == b or b == c or a == c:
    print("Sum = 0")
else:
    print("Sum =", a + b + c)
```

13) Write a Python program that will return True if the two given integer values are equal or their sum or difference is 5.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

if a == b or (a + b) == 5 or abs(a - b) == 5:
    print(True)
else:
    print(False)
```

14) Write a Python program to sum of the first n positive integers.

```
n = int(input("Enter a positive integer: "))

sum_n = n * (n + 1) // 2

print("Sum =", sum_n)
```

Alternative using loop:

```
n = int(input("Enter a positive integer: "))
total = 0

for i in range(1, n + 1):
    total += i

print("Sum =", total)
```

15) Write a Python program to calculate the length of a string.

```
string = input("Enter a string: ")

print("Length of string:", len(string))
```

16) Write a Python program to count the number of characters (character frequency) in a string.

```
string = input("Enter a string: ")

freq = {}

for char in string:
    freq[char] = freq.get(char, 0) + 1

print(freq)
```

Example:

Input:

hello

Output:

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

17) What are negative indexes and why are they used?

Negative indexing allows access to elements from the end of a sequence.

Example:

```
text = "Python"

print(text[-1]) # n
print(text[-2]) # o
print(text[-3]) # h
```

Why use negative indexing?

- Access elements from the end.
 - Useful when the length is unknown.
 - Simplifies string and list operations.
-

18) Write a Python program to count occurrences of a substring in a string.

```
string = input("Enter a string: ")
substring = input("Enter substring: ")

count = string.count(substring)

print("Occurrences:", count)
```

Example:

Input:

String: banana
Substring: an

Output:

Occurrences: 2

19) Write a Python program to count the occurrences of each word in a given sentence.

```
sentence = input("Enter a sentence: ")
words = sentence.split()
word_count = {}
for word in words:
    word_count[word] = word_count.get(word, 0) + 1
print(word_count)
```

Example:

Input:

this is a test this is

Output:

```
{'this': 2, 'is': 2, 'a': 1, 'test': 1}
```

20) Write a Python program to get a single string from two given strings, separated by a space and swap the first two characters of each string.

```
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")

new_str1 = str2[:2] + str1[2:]
new_str2 = str1[:2] + str2[2:]

result = new_str1 + " " + new_str2

print(result)
```

Example:

Input:

abc
xyz

Output:

xyc

abz

21) Write a Python program to add 'in' at the end of a given string (length should be at least 3). If the string already ends with 'ing', then add 'ly' instead. If the string length is less than 3, leave it unchanged.

```
string = input("Enter a string: ")

if len(string) < 3:
    result = string
elif string.endswith("ing"):
    result = string + "ly"
else:
    result = string + "ing"

print(result)
```

Example:

Input:

play

Output:

playing

22) Write a Python function to reverse a string if its length is a multiple of 4.

```
def reverse_string(s):
    if len(s) % 4 == 0:
        return s[::-1]
    return s

text = input("Enter a string: ")
print(reverse_string(text))
```

23) Write a Python program to get a string made of the first 2 and the last 2 characters from a given string. If the string length is less than 2, return an empty string.

```
string = input("Enter a string: ")

if len(string) < 2:
    print("")
else:
    print(string[:2] + string[-2:])
```

Example:

Input:

Python

Output:

Pyon

24) Write a Python function to insert a string in the middle of a string.

```
def insert_middle(main_string, insert_string):
    middle = len(main_string) // 2
    return main_string[:middle] + insert_string + main_string[middle:]

s1 = input("Enter main string: ")
s2 = input("Enter string to insert: ")

print(insert_middle(s1, s2))
```

Example:

Input:

HelloWorld
Python

Output:

HelloPythonWorld

25) What is List? How will you reverse a list?

List:

A list is an ordered, mutable collection of items in Python. It can store different data types.

Example:

```
my_list = [10, 20, 30, 40]
```

Reverse a List

Method 1: reverse()

```
my_list = [10, 20, 30, 40]
my_list.reverse()
print(my_list)
```

Output:

```
[40, 30, 20, 10]
```

Method 2: Slicing

```
my_list = [10, 20, 30, 40]
print(my_list[::-1])
```

26) How will you remove the last object from a list?

Using pop():

```
my_list = [10, 20, 30, 40]
my_list.pop()
print(my_list)
```

Output:

```
[10, 20, 30]
```

27) Suppose list1 = [2, 33, 222, 14, 25], what is list1[1]?

```
list1 = [2, 33, 222, 14, 25]
print(list1[1])
```

Output:

```
33
```

Explanation:

- Indexing starts from 0.

- `list1[1]` refers to the second element.

28) Differentiate between `append()` and `extend()` methods.

<code>append()</code>	<code>extend()</code>
Adds a single element to the end of the list	Adds multiple elements from an iterable
List size increases by 1	List size increases by the number of elements added
Entire object is added as one item	Elements are added individually

Example of `append()`

```
list1 = [1, 2, 3]
list1.append([4, 5])

print(list1)
```

Output:

```
[1, 2, 3, [4, 5]]
```

Example of `extend()`

```
list1 = [1, 2, 3]
list1.extend([4, 5])

print(list1)
```

Output:

```
[1, 2, 3, 4, 5]
```

29) Write a Python function to get the largest number, smallest number, and sum of all numbers from a list.

```
def list_info(numbers):
    largest = max(numbers)
    smallest = min(numbers)
    total = sum(numbers)

    return largest, smallest, total

nums = [10, 25, 5, 40, 15]

largest, smallest, total = list_info(nums)

print("Largest:", largest)
```

```
print("Smallest:", smallest)
print("Sum:", total)
```

Output:

```
Largest: 40
Smallest: 5
Sum: 95
```

30) How will you compare two lists?

Method 1: Using ==

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]

if list1 == list2:
    print("Lists are equal")
else:
    print("Lists are not equal")
```

Method 2: Compare after sorting

```
list1 = [3, 2, 1]
list2 = [1, 2, 3]

if sorted(list1) == sorted(list2):
    print("Lists contain same elements")
else:
    print("Lists are different")
```

Method 3: Using sets (ignores duplicates)

```
list1 = [1, 2, 3]
list2 = [3, 2, 1]

if set(list1) == set(list2):
    print("Same unique elements")
```

31) Write a Python program to count the number of strings where the string length is 2 or more and the first and last character are the same from a given list of strings.

```
def count_strings(words):
    count = 0

    for word in words:
        if len(word) >= 2 and word[0] == word[-1]:
            count += 1

    return count

sample_list = ['abc', 'xyz', 'aba', '1221']
print(count_strings(sample_list))
```

Output:

2

32) Write a Python program to remove duplicates from a list.

Method 1: Using set()

```
numbers = [1, 2, 2, 3, 4, 4, 5]

unique_numbers = list(set(numbers))

print(unique_numbers)
```

Method 2: Preserve Order

```
numbers = [1, 2, 2, 3, 4, 4, 5]
result = []

for item in numbers:
    if item not in result:
        result.append(item)

print(result)
```

Output:

[1, 2, 3, 4, 5]

33) Write a Python program to check a list is empty or not.

```
my_list = []

if not my_list:
    print("List is empty")
else:
    print("List is not empty")
```

34) Write a Python function that takes two lists and returns True if they have at least one common member.

```
def common_member(list1, list2):
    for item in list1:
        if item in list2:
            return True
    return False

list1 = [1, 2, 3, 4]
list2 = [5, 6, 3, 8]

print(common_member(list1, list2))
```

Output:

True

35) Write a Python program to generate and print a list of first and last 5 elements where the values are squares of numbers between 1 and 30.

```
squares = [x**2 for x in range(1, 31)]

print("First 5 elements:", squares[:5])
print("Last 5 elements:", squares[-5:])
```

Output:

```
First 5 elements: [1, 4, 9, 16, 25]
Last 5 elements: [676, 729, 784, 841, 900]
```

36) Write a Python function that takes a list and returns a new list with unique elements of the first list.

```
def unique_list(lst):
    unique = []

    for item in lst:
        if item not in unique:
            unique.append(item)

    return unique

print(unique_list([1, 2, 2, 3, 4, 4, 5]))
```

Output:

```
[1, 2, 3, 4, 5]
```

37) Write a Python program to convert a list of characters into a string.

```
characters = ['P', 'Y', 'T', 'H', 'O', 'N']  
string = ''.join(characters)  
print(string)
```

Output:

```
PYTHON
```

38) Write a Python program to select an item randomly from a list.

```
import random  
items = ['Apple', 'Banana', 'Orange', 'Mango']  
print(random.choice(items))
```

39) Write a Python program to find the second smallest number in a list.

```
numbers = [10, 5, 20, 8, 3]  
unique_numbers = sorted(set(numbers))  
print("Second smallest:", unique_numbers[1])
```

Output:

```
Second smallest: 5
```

Alternative

```
numbers = [10, 5, 20, 8, 3]  
  
smallest = min(numbers)  
numbers.remove(smallest)  
  
print("Second smallest:", min(numbers))
```

40) Write a Python program to get unique values from a list.

```
numbers = [1, 2, 2, 3, 4, 4, 5]

unique_values = list(set(numbers))

print(unique_values)
Preserving Order
numbers = [1, 2, 2, 3, 4, 4, 5]
unique = []

for item in numbers:
    if item not in unique:
        unique.append(item)

print(unique)
```

Output:

```
[1, 2, 3, 4, 5]
```

41) Write a Python program to check whether a list contains a sublist.

```
def contains_sublist(main_list, sub_list):
    for item in main_list:
        if isinstance(item, list) and item == sub_list:
            return True
    return False

main_list = [1, 2, [3, 4], 5]
sub_list = [3, 4]

print(contains_sublist(main_list, sub_list))
```

Output:

```
True
```

42) Write a Python program to split a list into different variables.

```
my_list = [10, 20, 30]

a, b, c = my_list

print(a)
print(b)
print(c)
```

Output:

10
20
30

43) What is a tuple? Difference between list and tuple.

Tuple

A tuple is an ordered collection of elements that is **immutable** (cannot be changed after creation).

Example:

```
my_tuple = (10, 20, 30)
```

Difference Between List and Tuple

List	Tuple
Mutable	Immutable
Uses []	Uses ()
Slower	Faster
More memory	Less memory
Can be modified	Cannot be modified

Example:

```
my_list = [1, 2, 3]  
my_tuple = (1, 2, 3)
```

44) Write a Python program to create a tuple with different data types.

```
my_tuple = (10, "Python", 3.14, True)  
  
print(my_tuple)
```

Output:


```
(10, 'Python', 3.14, True)
```

45) Write a Python program to unzip a list of tuples into individual lists.

```
data = [(1, 'A'), (2, 'B'), (3, 'C')]

numbers, letters = zip(*data)

print(list(numbers))
print(list(letters))
```

Output:

```
[1, 2, 3]
['A', 'B', 'C']
```

46) Write a Python program to convert a list of tuples into a dictionary.

```
data = [('a', 1), ('b', 2), ('c', 3)]

result = dict(data)

print(result)
```

Output:

```
{'a': 1, 'b': 2, 'c': 3}
```

47) How will you create a dictionary using tuples in Python?

```
tuples = (('name', 'John'),
          ('age', 25),
          ('city', 'Ahmedabad'))

dictionary = dict(tuples)

print(dictionary)
```

Output:

```
{'name': 'John', 'age': 25, 'city': 'Ahmedabad'}
```

48) Write a Python script to sort (ascending and descending) a dictionary by value.

```
my_dict = {'a': 5, 'b': 2, 'c': 8, 'd': 1}
```

```

ascending = dict(sorted(my_dict.items(), key=lambda item: item[1]))

descending = dict(sorted(my_dict.items(),
                        key=lambda item: item[1],
                        reverse=True))

print("Ascending:", ascending)
print("Descending:", descending)

```

Output:

```

Ascending: {'d': 1, 'b': 2, 'a': 5, 'c': 8}
Descending: {'c': 8, 'a': 5, 'b': 2, 'd': 1}

```

49) Write a Python script to concatenate dictionaries to create a new one.

```

dict1 = {1: 10, 2: 20}
dict2 = {3: 30, 4: 40}
dict3 = {5: 50, 6: 60}

result = {}

result.update(dict1)
result.update(dict2)
result.update(dict3)

print(result)

```

Alternative (Python 3.5+)

```

result = {**dict1, **dict2, **dict3}
print(result)

```

Output:

```
{1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
```

50) Write a Python script to check if a given key already exists in a dictionary.

```

my_dict = {
    'name': 'John',
    'age': 25,
    'city': 'Ahmedabad'
}

key = input("Enter key to search: ")

if key in my_dict:
    print("Key exists")
else:
    print("Key does not exist")

```

Example

Input:

age

Output:

Key exists

51) How Do You Traverse Through a Dictionary Object in Python?

A dictionary can be traversed using loops.

Traverse Keys

```
my_dict = {'a': 1, 'b': 2, 'c': 3}

for key in my_dict:
    print(key)
```

Traverse Values

```
for value in my_dict.values():
    print(value)
```

Traverse Key-Value Pairs

```
for key, value in my_dict.items():
    print(key, ":", value)
```

Output:

```
a : 1
b : 2
c : 3
```

52) How Do You Check the Presence of a Key in a Dictionary?

Using the `in` operator:

```
my_dict = {'name': 'John', 'age': 25}

if 'age' in my_dict:
    print("Key exists")
else:
    print("Key does not exist")
```

Output:

Key exists

53) Write a Python script to print a dictionary where the keys are numbers between 1 and 15.

```
my_dict = {}

for x in range(1, 16):
    my_dict[x] = x * x

print(my_dict)
```

Output:

```
{
1:1, 2:4, 3:9, 4:16, 5:25,
6:36, 7:49, 8:64, 9:81,
10:100, 11:121, 12:144,
13:169, 14:196, 15:225
}
```

54) Write a Python program to check multiple keys exist in a dictionary.

```
student = {
    'name': 'John',
    'age': 20,
    'city': 'Ahmedabad'
}

keys = ['name', 'age']

if all(key in student for key in keys):
    print("All keys exist")
else:
    print("Some keys are missing")
```

Output:

All keys exist

55) Write a Python script to merge two Python dictionaries.

```
dict1 = {'a': 100, 'b': 200}
dict2 = {'c': 300, 'd': 400}

merged = {**dict1, **dict2}

print(merged)
```

Alternative

```
merged = dict1.copy()
merged.update(dict2)
```

Output:

```
{'a': 100, 'b': 200, 'c': 300, 'd': 400}
```

56) Write a Python program to map two lists into a dictionary.

```
keys = ['a', 'b', 'c', 'd']
values = [400, 400, 300, 400]

result = dict(zip(keys, values))

print(result)
```

Output:

```
{'a': 400, 'b': 400, 'c': 300, 'd': 400}
```

Using Counter

```
from collections import Counter

keys = ['a', 'b', 'c', 'd']
values = [400, 400, 300, 400]

result = Counter(dict(zip(keys, values)))

print(result)
```

57) Write a Python program to find the highest 3 values in a dictionary.

```
my_dict = {
    'a': 50,
    'b': 80,
    'c': 20,
    'd': 90,
    'e': 60
}

highest = sorted(my_dict.values(), reverse=True)[:3]
```

```
print("Highest 3 values:", highest)
```

Output:

```
Highest 3 values: [90, 80, 60]
```

58) Write a Python program to combine values in a list of dictionaries.

Sample Data

```
data = [
    {'item': 'item1', 'amount': 400},
    {'item': 'item2', 'amount': 300},
    {'item': 'item1', 'amount': 750}
]
```

Program

```
from collections import Counter

data = [
    {'item': 'item1', 'amount': 400},
    {'item': 'item2', 'amount': 300},
    {'item': 'item1', 'amount': 750}
]

counter = Counter()

for d in data:
    counter[d['item']] += d['amount']

print(counter)
```

Output:

```
Counter({'item1': 1150, 'item2': 300})
```

59) Write a Python program to create a dictionary from a string. Track the count of letters.

```
string = input("Enter a string: ")

result = {}

for char in string:
    result[char] = result.get(char, 0) + 1

print(result)
```

Example

Input:

python

Output:

```
{'p':1, 'y':1, 't':1, 'h':1, 'o':1, 'n':1}
```

60) Sample string: 'w3resource'

Expected Output:

```
{'w': 1,  
'3': 1,  
'r': 2,  
'e': 2,  
's': 1,  
'o': 1,  
'u': 1,  
'c': 1}
```

Program

```
string = "w3resource"  
  
result = {}  
  
for char in string:  
    result[char] = result.get(char, 0) + 1  
  
print(result)
```

Output:

```
{'w': 1,  
'3': 1,  
'r': 2,  
'e': 2,  
's': 1,  
'o': 1,  
'u': 1,  
'c': 1}
```

61) Python function to calculate the factorial of a number

```
def factorial(n):  
    if n < 0:  
        return "Factorial not defined for negative numbers"  
    result = 1  
    for i in range(1, n + 1):
```

```
        result *= i
    return result

print(factorial(5))
```

62) Python function to check whether a number is in a given range

```
def check_range(num, start, end):
    if start <= num <= end:
        return True
    return False

print(check_range(5, 1, 10))
```

63) Python function to check whether a number is perfect or not

```
def is_perfect(num):
    total = 0
    for i in range(1, num):
        if num % i == 0:
            total += i
    return total == num

print(is_perfect(28))
```

64) Python function to check whether a string is palindrome or not

```
def is_palindrome(text):
    return text == text[::-1]

print(is_palindrome("madam"))
```

65) How many basic types of functions are available in Python?

There are **2 basic types of functions**:

1. **Built-in Functions** – Predefined functions such as `print()`, `len()`, `type()`.
 2. **User-defined Functions** – Functions created by the programmer using `def`.
-

66) How can you pick a random item from a list or tuple?

```
import random
```



```
items = [10, 20, 30, 40]
print(random.choice(items))
```

67) How can you pick a random item from a range?

```
import random

print(random.choice(range(1, 11)))
```

68) How can you get a random number in Python?

```
import random

print(random.randint(1, 100))    # Integer
print(random.random())           # Float between 0 and 1
```

69) How will you set the starting value in generating random numbers?

Use `seed()`.

```
import random

random.seed(10)
print(random.randint(1, 100))
```

70) How will you randomize the items of a list in place?

```
import random

numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers)

print(numbers)
```

71) What is File function in Python? What are keywords to create and write file?

Python uses the `open()` function to create, read, write, and manipulate files.

Modes:

- `r` → Read
- `w` → Write (creates file if not exists)
- `a` → Append
- `x` → Create file

- `r+` → Read and Write

Example:

```
file = open("sample.txt", "w")
file.write("Hello World")
file.close()
```

72) Python program to read an entire text file

```
with open("sample.txt", "r") as file:
    data = file.read()

print(data)
```

73) Python program to append text to a file and display the text

```
with open("sample.txt", "a") as file:
    file.write("\nNew Text Added")

with open("sample.txt", "r") as file:
    print(file.read())
```

74) Python program to read first n lines of a file

```
n = 3

with open("sample.txt", "r") as file:
    for i in range(n):
        print(file.readline(), end="")
```

75) Python program to read last n lines of a file

```
n = 3

with open("sample.txt", "r") as file:
    lines = file.readlines()

for line in lines[-n:]:
    print(line, end="")
```

76) Python program to read a file line by line and store it into a list

```
with open("sample.txt", "r") as file:
    lines = file.readlines()
```

```
print(lines)
```

77) Python program to read a file line by line and store it into a variable

```
with open("sample.txt", "r") as file:
    content = ""

    for line in file:
        content += line

print(content)
```

78) Python program to find the longest words

```
with open("sample.txt", "r") as file:
    words = file.read().split()

max_len = len(max(words, key=len))

longest_words = [word for word in words if len(word) == max_len]

print(longest_words)
```

79) Python program to count the number of lines in a text file

```
with open("sample.txt", "r") as file:
    count = len(file.readlines())

print("Number of lines:", count)
```

80) Python program to count the frequency of words in a file

```
from collections import Counter

with open("sample.txt", "r") as file:
    words = file.read().split()

frequency = Counter(words)

print(frequency)
```

81) Python program to write a list to a file

```
items = ["Apple", "Banana", "Mango"]
```

```
with open("sample.txt", "w") as file:
    for item in items:
        file.write(item + "\n")
```

82) Python program to copy contents of one file to another

```
with open("source.txt", "r") as source:
    data = source.read()

with open("destination.txt", "w") as destination:
    destination.write(data)

print("File copied successfully.")
```

83) Explain Exception Handling. What is an Error in Python?

Error: A problem in a program that causes it to stop execution.

Examples:

- SyntaxError
- NameError
- TypeError
- ZeroDivisionError

Exception Handling: A mechanism to handle runtime errors gracefully using `try`, `except`, `else`, and `finally`.

Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

84) How many except statements can a try-except block have? Name some built-in exception classes.

A `try` block can have **multiple except blocks**.

Example:

```
try:
    pass
except ValueError:
    pass
```

```
except TypeError:
    pass
```

Built-in Exceptions:

- ValueError
- TypeError
- NameError
- IndexError
- KeyError
- ZeroDivisionError
- FileNotFoundError
- ImportError

85) When will the else part of try-except-else be executed?

The `else` block executes **only if no exception occurs** in the `try` block.

```
try:
    num = 10 / 2
except ZeroDivisionError:
    print("Error")
else:
    print("Success")
```

86) Can one block of except statements handle multiple exceptions?

Yes.

```
try:
    x = int("abc")
except (ValueError, TypeError):
    print("Handled multiple exceptions")
```

87) When is the finally block executed?

The `finally` block executes **always**, whether an exception occurs or not.

```
try:
    print("Try block")
except:
    print("Exception")
finally:
    print("Finally block")
```

88) What happens when '1' == 1 is executed?

```
print('1' == 1)
```

Output:

False

Reason: '1' is a string and 1 is an integer. Different data types are not equal.

89) How do you handle exceptions with Try/Except/Finally in Python?

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Invalid input")

finally:
    print("Program ended")
```

Explanation:

- try → Code that may cause an exception.
 - except → Handles the exception.
 - finally → Executes regardless of exception.
-

```
try:
    num = int(input("Enter an odd number: "))

    if num % 2 == 0:
        raise ValueError("Even number entered!")

    print("You entered an odd number:", num)

except ValueError as e:
    print("Exception:", e)
```

Output Example:

```
Enter an odd number: 8
Exception: Even number entered!
```

